

Week 4

1. Graphical User Interface Design Tutorial: Menus and GridBagLayout

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Monday, January 26th, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

Agenda



Pull-down Menus

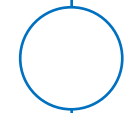
Icons in Menu Items

**Checkboxes and Radio Buttons in
Menu Items**

Pop-up Menus

**Keyboard Shortcuts and
Mnemonics**

GridBagLayout

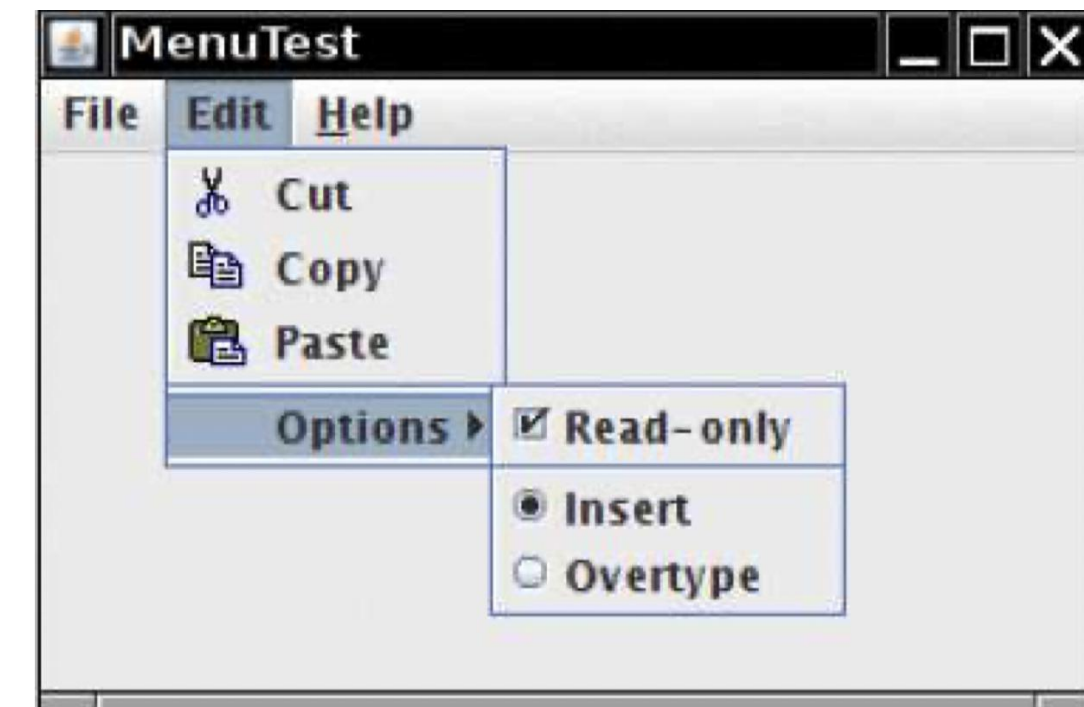


Pull-down Menus

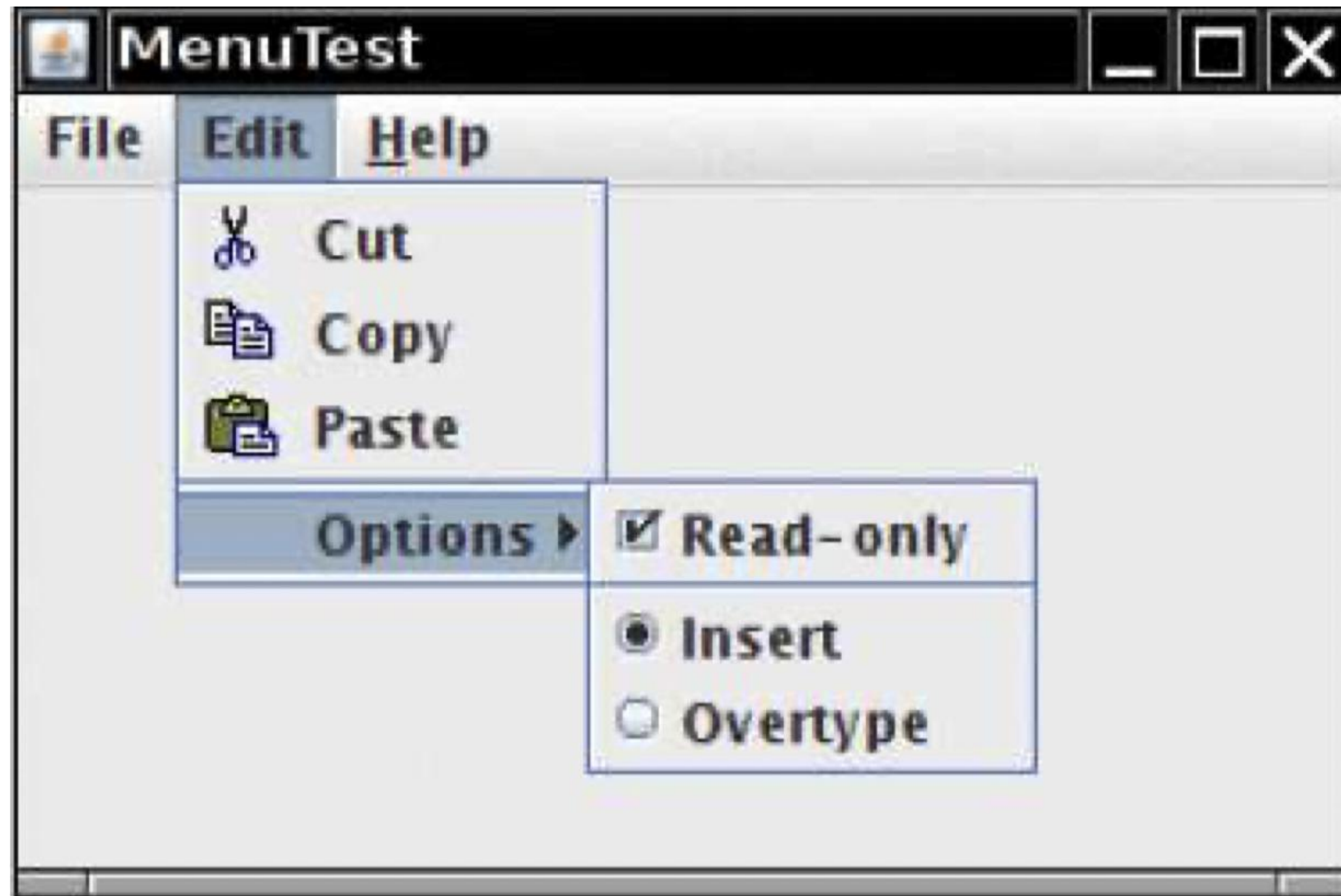


Pull-down Menus

- Pull-down menus are popular user interface elements that Java Swing supports.
- A menu bar at the top of the window containing the names of pull-down menus.
- Clicking on a name opens the menu containing menu items and submenus.
- When a user clicks on a menu item, all menus are closed, and a message is sent to the program.



Pull-down Menus



Building Menus

- To create menus, we need to create a menu bar first
`JMenuBar menuBar = new JMenuBar();`
- Add menu bar to the top of a frame
`frame.setJMenuBar(menuBar);`
- For each menu, we need to create the menu object first
`JMenu fileMenu = new JMenu("File");`
- Add the top-level menus to the menu bar
`menubar.add(fileMenu)`



Building Menus

- Add menu items, separators and submenus to the menu object

```
JMenuItem saveItem = new JMenuItem("Save");  
fileMenu.add(pasteItem);  
fileMenu.addSeparator();  
JMenu optionsMenu = . . .; // a JMenu object  
fileMenu.add(optionsMenu);
```



Handling Menu Item Selection Event

- When a user selects a menu item, an action event is triggered.
- To detect and handle menu item selection event, you need to install an ActionListener for each menu item.

```
ActionListener listener = . . . ;  
pasteItem.addActionListener(listener);
```

- The menu JMenuItem.add(String s) adds a menu item at the end of a menu and returns the created menu item.

```
JMenuItem pasteItem = editMenu.add("Paste");  
pasteItem.addActionListener(listener);
```


Handling Menu Item Selection Event

- It is also possible to implements the Action interface, usually by extending the AbstractAction class and override the actionPerformed method to hold the menu action handler.
- We can then add the action to the menu

```
JMenuItem exitItem = fileMenu.add(exitAction);
```

This can be achieved as follows also:

```
JMenuItem exitItem = new JMenuItem(exitAction);  
fileMenu.add(exitItem);
```



Handling Menu Item Selection Event

```
var exitAction = new AbstractAction("Exit") // menu item text
goes here
{
    public void actionPerformed(ActionEvent event)
    {
        // action code goes here
        System.exit(0);
    }
};
```



Important Methods in Javax.swing.JMenu

Method	Description
<code>JMenu(String label)</code>	constructs a menu with the given label
<code>JMenuItem add(JMenuItem item)</code>	adds a menu item (or a menu)
<code>JMenuItem add(String label)</code>	adds a menu item with the given label to this menu and returns the item
<code>JMenuItem add(Action a)</code>	adds a menu item with the given action to this menu and returns the item
<code>void addSeparator()</code>	adds a separator line to the menu
<code>JMenuItem insert(JMenuItem menu, int index)</code>	adds a new menu item (or submenu) to the menu at a specific index
<code>JMenuItem insert(Action a, int index)</code>	adds a new menu item with the given action at a specific index
<code>void insertSeparator(int index)</code>	adds a separator to the menu
<code>void remove(int index)</code>	removes a specific item from the menu
<code>void remove(JMenuItem item)</code>	removes a specific item from the menu

Important Methods in `javax.swing.JMenuItem`

Method	Description
<code>JMenuItem(String label)</code>	constructs a menu item with a given label
<code>JMenuItem(Action a)</code>	constructs a menu item for the given action

Menu Related Methods in `javax.swing.AbstractButton`

Method	Description
<code>void setAction(Action a)</code>	sets the action for this button or menu item

Menu Related Methods in `javax.swing.JFrame`

Method	Description
<code>void setJMenuBar(JMenuBar menubar)</code>	sets the menu bar for this frame

Icons in Menu Items



Icons in Menu Item

- Menu items are very similar to buttons. Just like buttons, they can have text labels, icons or both.

```
ImageIcon imageIcon = new ImageIcon("cut.gif");  
JMenuItem cutItem = new JMenuItem("Cut", imageIcon);
```

- By default, the menu item text is placed to the right of the icon. If you prefer the text to be placed on the left, call the `setHorizontalTextPosition` method that the `JMenuItem` class inherits from the `AbstractButton` class.

```
cutItem.setHorizontalTextPosition(SwingConstants.LEFT);
```

Creating Menu Item from Action

- When we construct a menu item out of an action, the `Action.NAME` value becomes the text of the menu item and the `Action.SMALL_ICON` value becomes the icon.

```
cutAction = new
    AbstractAction("Cut", new ImageIcon("cut.gif"))
    {
        public void actionPerformed(ActionEvent event)
        {
            . . .
        }
    };
```



Checkbox and Radio Buttons in Menu Items



Checkbox and Radio Button Menu Items

- We can also include checkboxes and radio buttons as menu items

```
JCheckBoxMenuItem readonlyItem;  
readonlyItem = new JCheckBoxMenuItem("Read-only");  
optionsMenu.add(readonlyItem);
```



Checkbox and Radio Button Menu Items

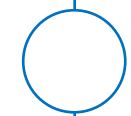
- We can also include checkboxes and radio buttons as menu items

```
JCheckBoxMenuItem readonlyItem;  
readonlyItem = new JCheckBoxMenuItem("Read-only");  
optionsMenu.add(readonlyItem);
```

- The radio button menu items work just like regular radio buttons. However, we need to put them into a button group.

```
ButtonGroup group = new ButtonGroup();  
JRadioButtonMenuItem insertItem;  
insertItem = new JRadioButtonMenuItem("Insert");  
insertItem.setSelected(true);  
  
JRadioButtonMenuItem overtypeItem;  
overtimeItem = new JRadioButtonMenuItem("Overtime");  
  
group.add(insertItem);  
group.add(overtimeItem);  
optionsMenu.add(insertItem);  
optionsMenu.add(overtimeItem);
```



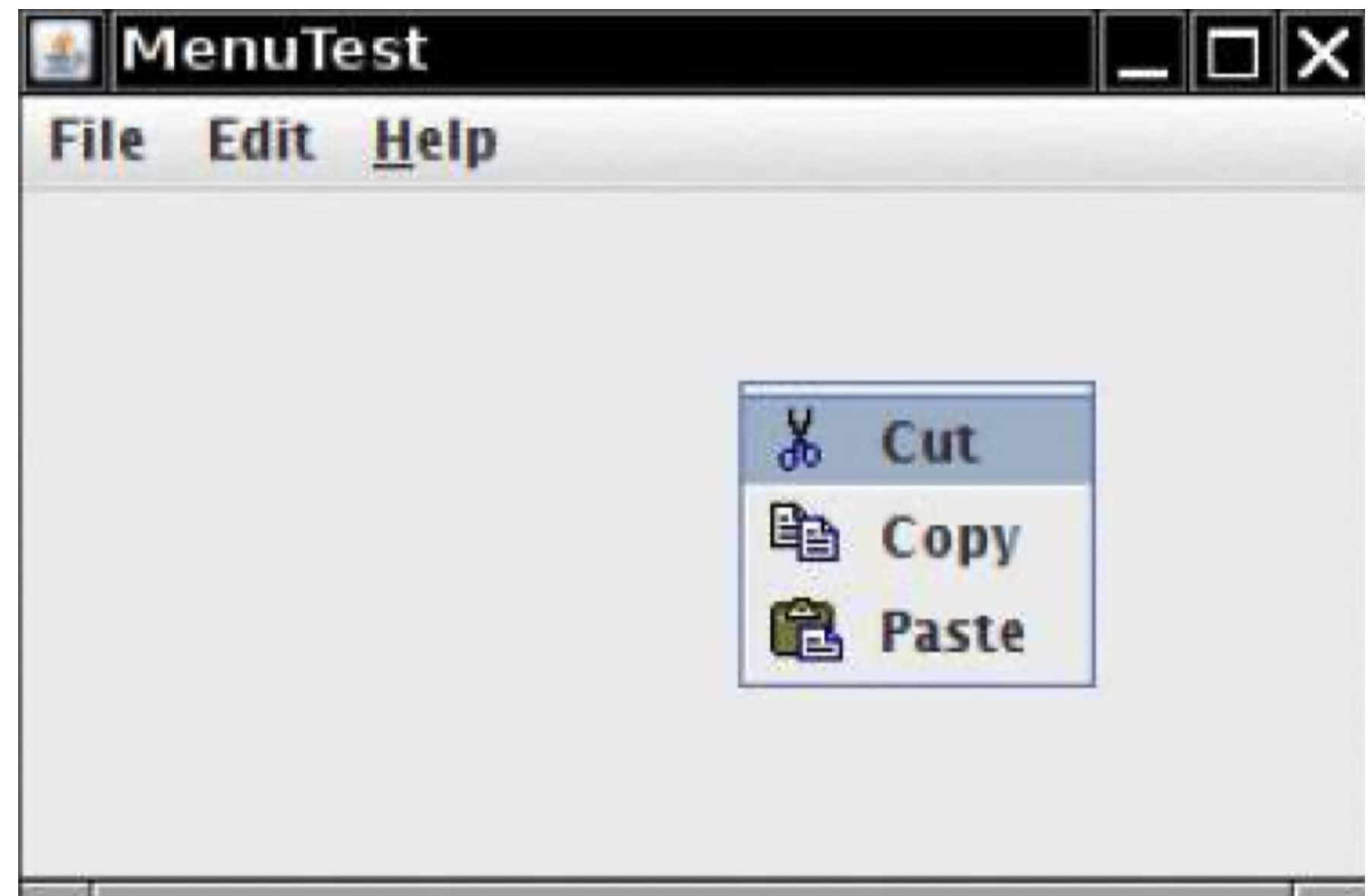


Pop-up Menus



Pop-Up Menus

- A pop-up menu is a menu that is not attached to a menu bar but floats somewhere.

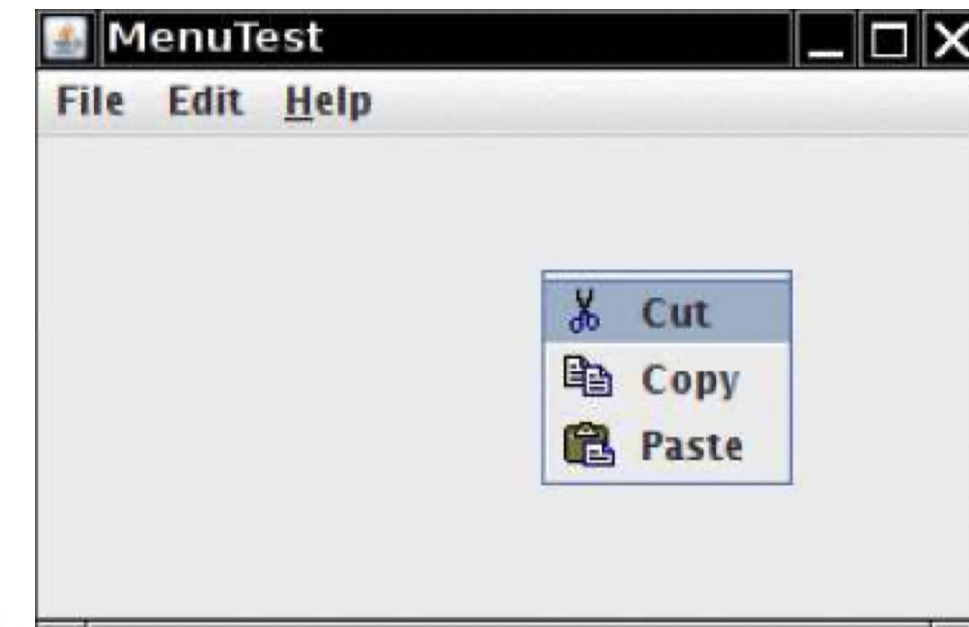


Pop-Up Menus

- A pop-up menu is created just like a regular menu, except, it has no title:

```
JPopupMenu popup = new JPopupMenu();  
JMenuItem item = new JMenuItem("Cut");  
item.addActionListener(listener);  
popup.add(item);
```
- Typically, we pop-up a menu when a user clicks a particular mouse button (also known as the pop-up trigger).
- For example, a pop-up menu is triggered when a users clicks the right mouse button.
- A pop-menu is shown by specifying the parent component and the location of the pop-up menu.

```
popup.show(panel, x, y);
```



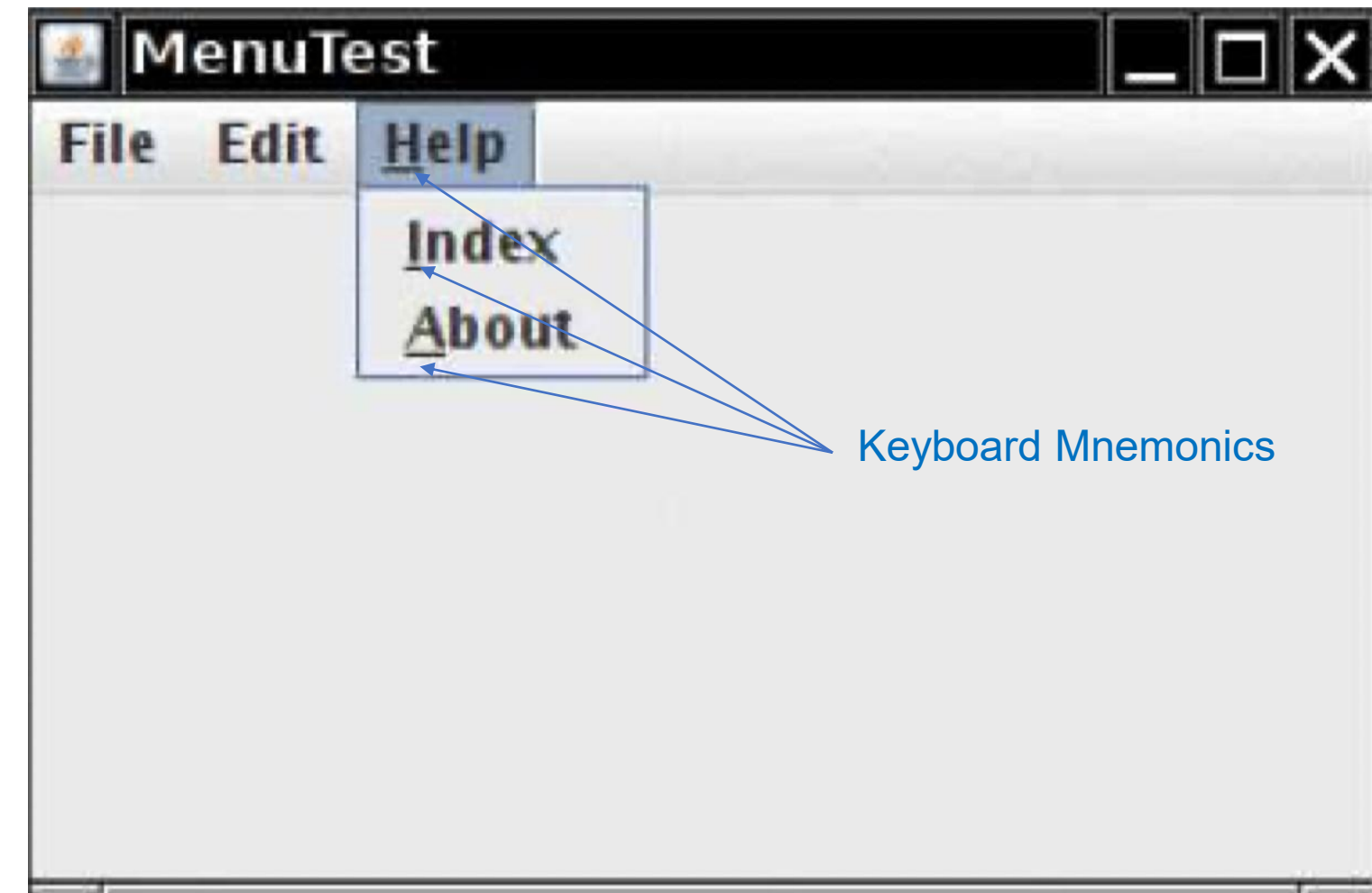
Keyboard Mnemonics and Accelerators



Keyboard Mnemonics and Accelerators

- Experienced user can select menu items using keyboard shortcuts or keyboard mnemonics.

```
JMenuItem indexItem = new JMenuItem("Index", 'I');  
JMenuItem aboutItem = new JMenuItem("About", 'A');
```



Keyboard Mnemonics and Accelerators

- What if I don't what to underline the first letter of the menu item that matches the mnemonic?



Keyboard Mnemonics and Accelerators

- What if I don't want to underline the first letter of the menu item that matches the mnemonic?
- For example, if you have a mnemonic 'A' for the menu item "Save As," then it makes more sense to underline the second 'A' (Save As):
- `use setDisplayedMnemonicIndex method`



Keyboard Mnemonics and Accelerators

- What if I don't what to underline the first letter of the menu item that matches the mnemonic?
- For example, if you have a mnemonic 'A' for the menu item "Save As," then it makes more sense to underline the second 'A' (Save As):

- `use setDisplayedMnemonicIndex method`

- What if I have an Action object?

`aboutAction.putValue(Action.MNEMONIC_KEY, Integer.valueOf('A'));`



Keyboard Mnemonics and Accelerators

- What if I don't want to underline the first letter of the menu item that matches the mnemonic?
- For example, if you have a mnemonic 'A' for the menu item "Save As," then it makes more sense to underline the second 'A' (Save As):

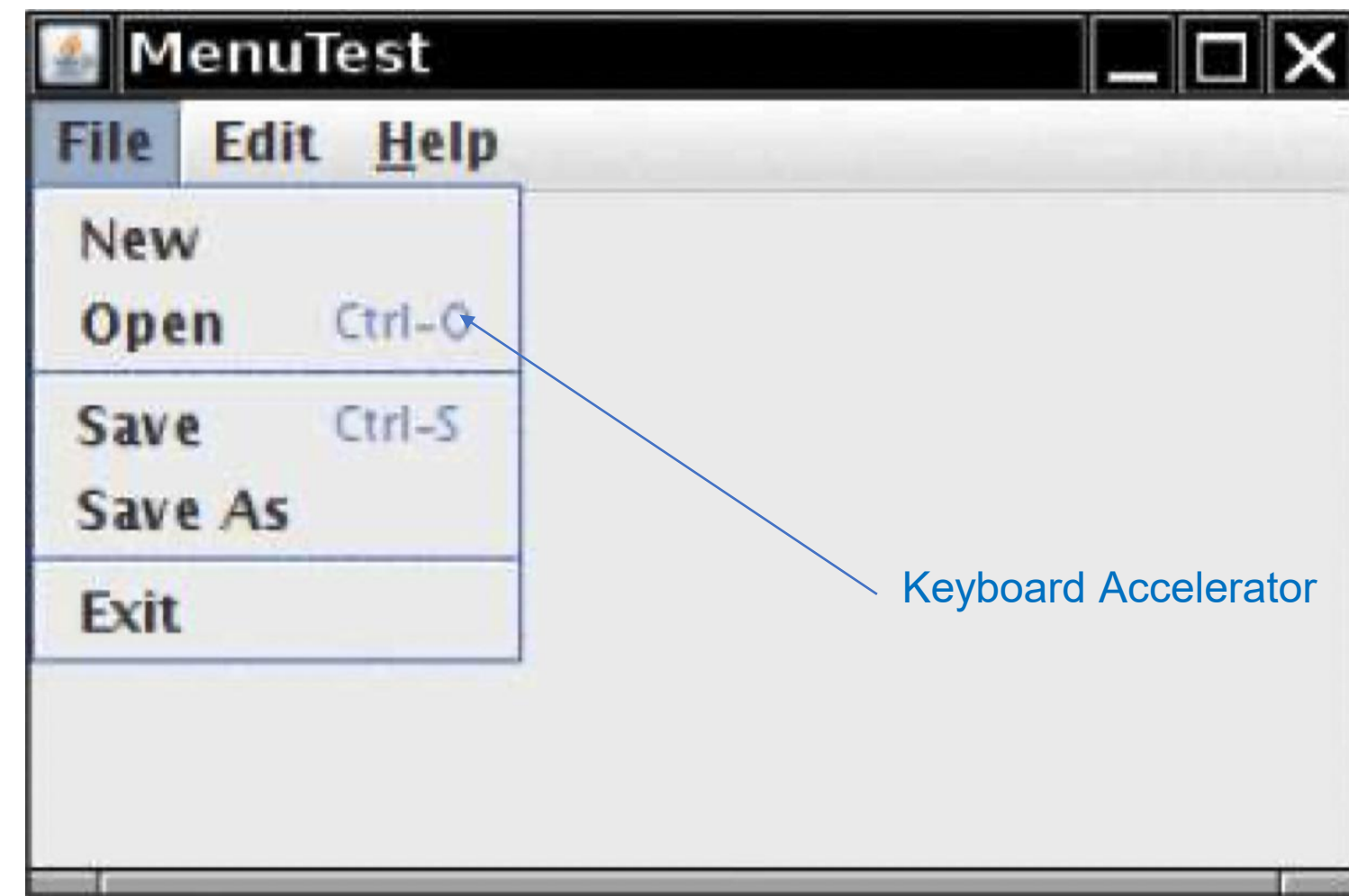
- - use `setDisplayMnemonicIndex` method

- What if I have an Action object?
 - `aboutAction.putValue(Action.MNEMONIC_KEY, Integer.valueOf('A'));`
- We can only supply mnemonic letter to the constructor of a menu item. To attach mnemonic to a menu, we need to call the following method:
 - `JMenu helpMenu = new JMenu("Help");`
 - `helpMenu.setMnemonic('H');`



Keyboard Mnemonics and Accelerators

- Accelerators are keyboard shortcuts that can allow you to select a menu item without even opening the menu.
- For example, Ctrl+O and Ctrl+S are two popular keyboard accelerators for opening and saving files: `openItem.setAccelerator(KeyStroke.getKeyStroke("ctrl O"));`



Toolbars



Toolbar

- A toolbar shows a group of buttons on a bar that represents the frequently used commands.



Toolbar

- You can create a toolbar as follows:

```
JToolBar toolbar = new JToolBar();  
toolbar.add(blueButton);
```
- The JToolBar class also has a method to add an Action object

```
toolbar.add(blueAction);
```
- You can separate the group of buttons using a separator:

```
toolbar.addSeparator();
```
- We can add the toolbar to the frame as follows:

```
add(toolbar, BorderLayout.NORTH);
```


Toolbar

- By default, toolbars are shown horizontally. To show a toolbar vertically use the following method:

```
toolbar = new JToolBar(SwingConstants.VERTICAL)
```

or

```
toolbar = new JToolBar(titleString, SwingConstants.VERTICAL)
```



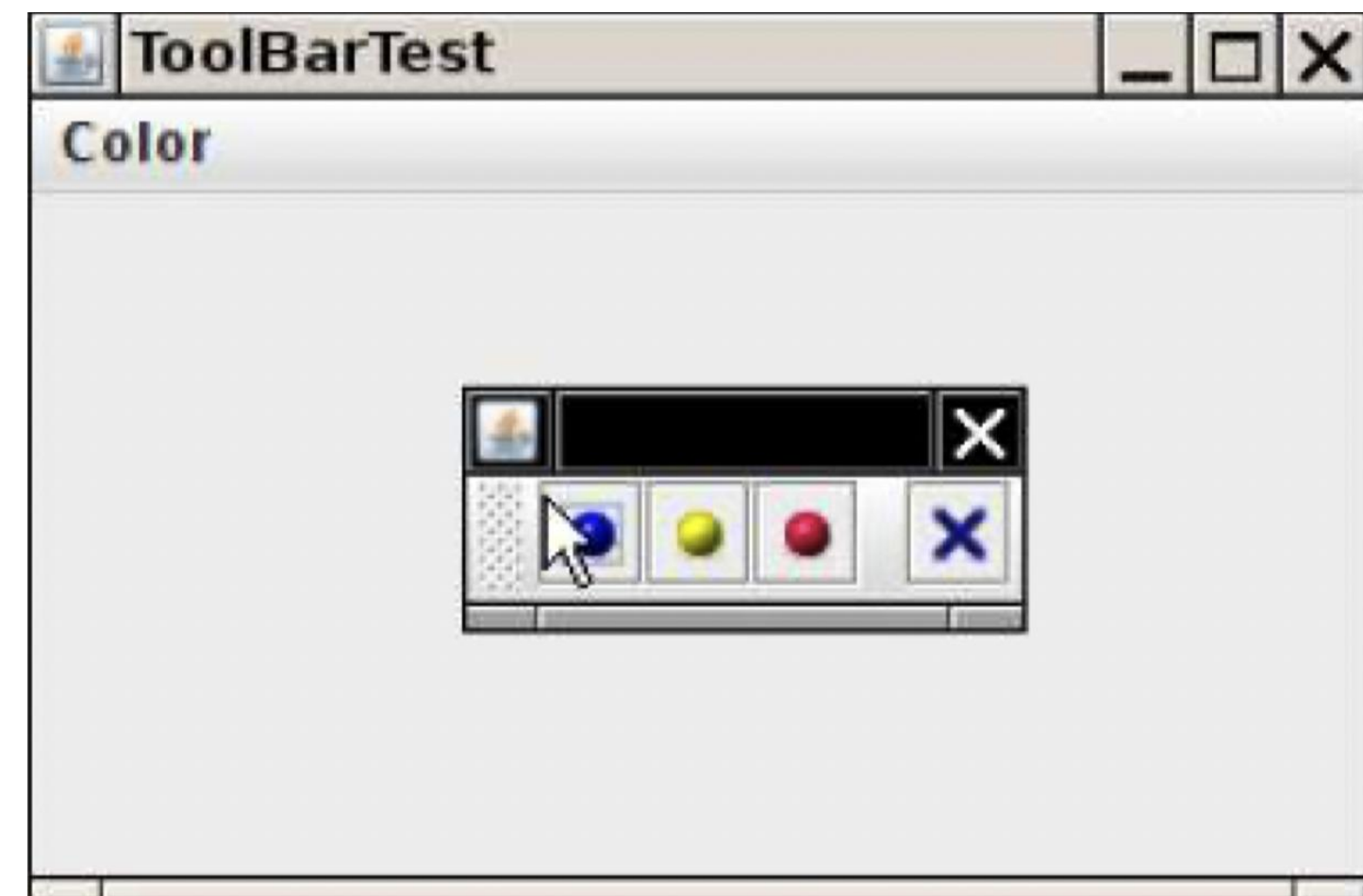
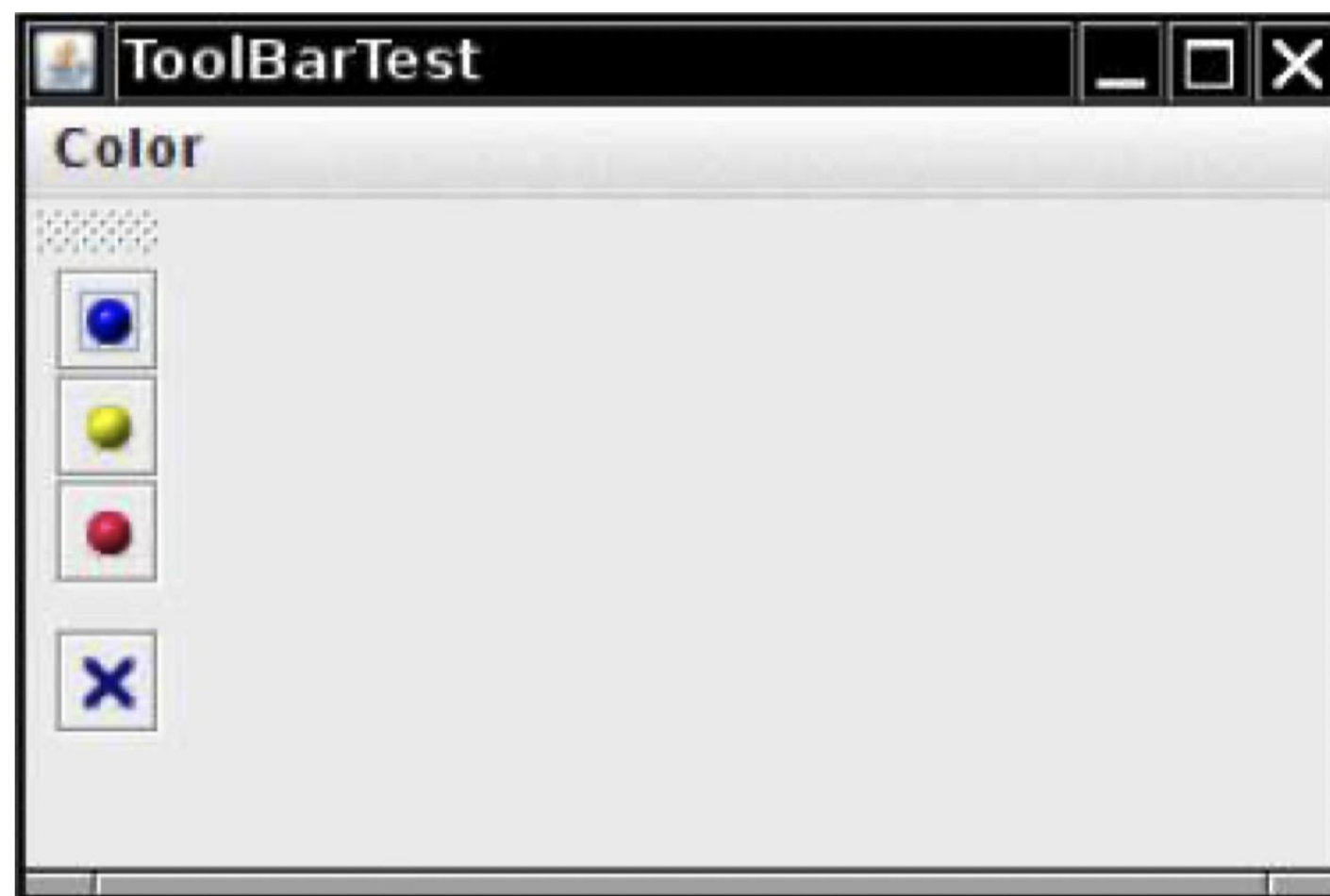
Toolbar

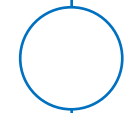
- By default, toolbars are shown horizontally. To show a toolbar vertically use the following method:

```
toolbar = new JToolBar(SwingConstants.VERTICAL)
```

or

```
toolbar = new JToolBar(titleString, SwingConstants.VERTICAL)
```

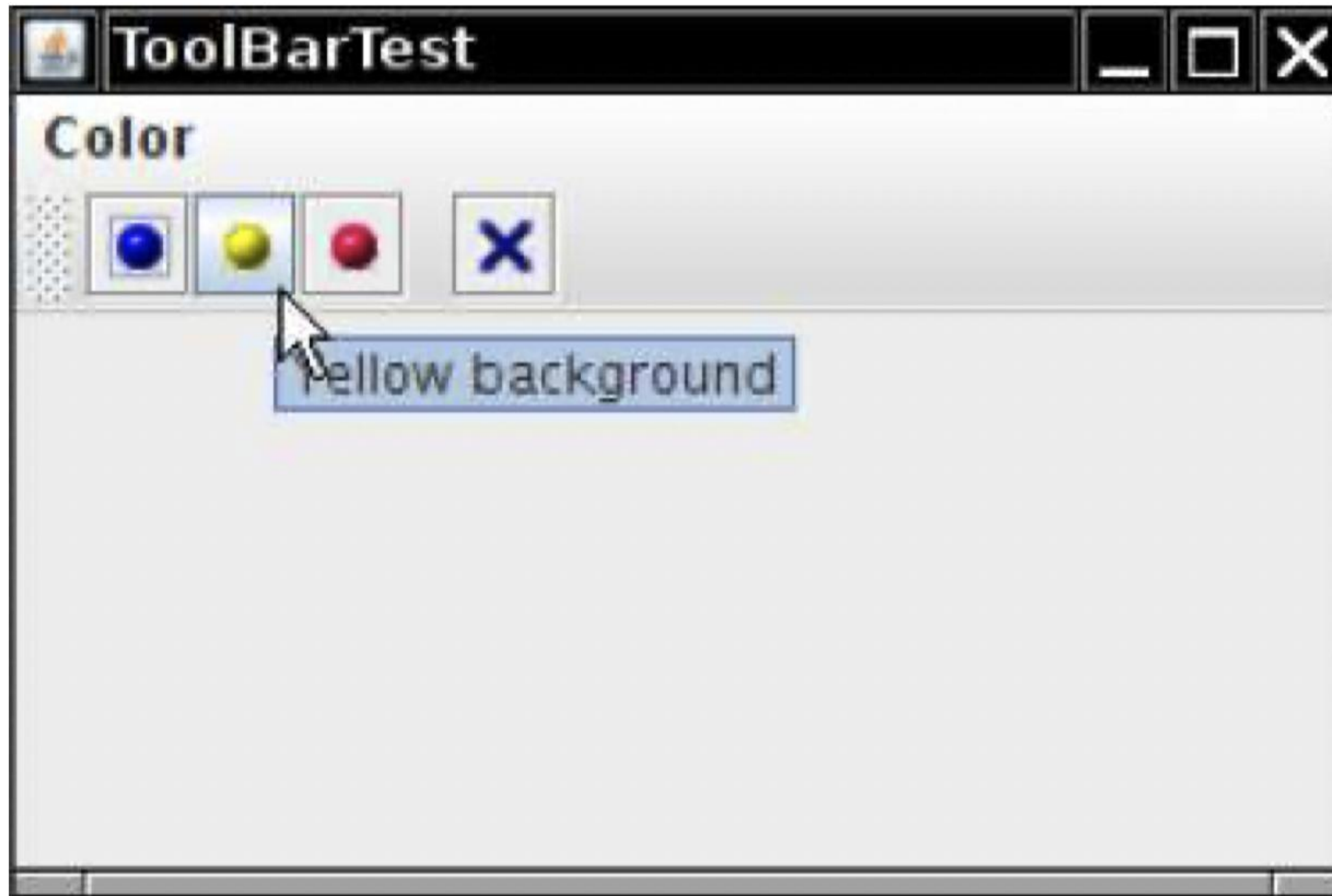




Tooltips



Adding Tooltips



Adding Tooltips

- In Swing, you can add tooltips to any JComponent simply by calling the `setToolTipText` method:

```
exitButton.setToolTipText("Exit");
```

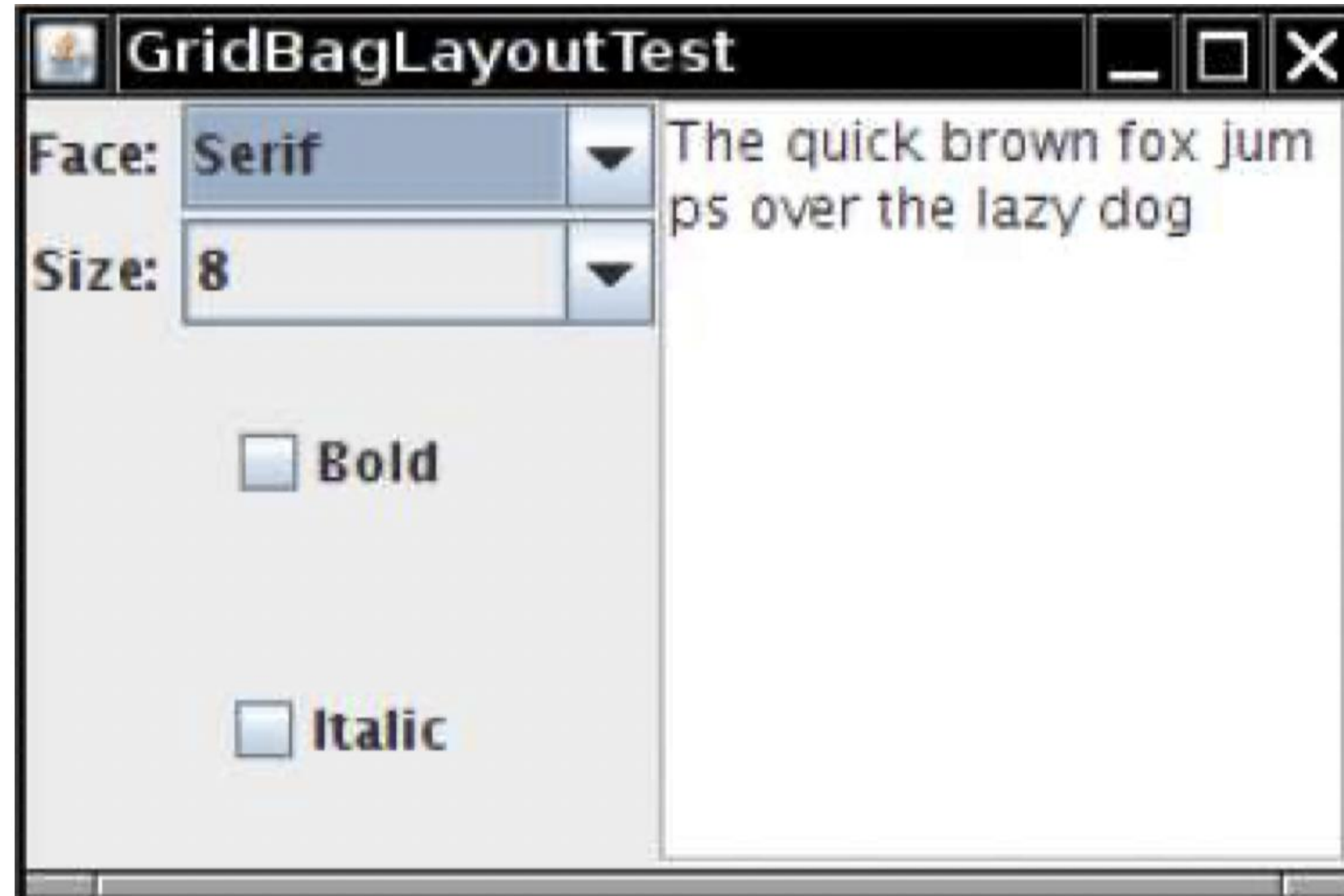
- Alternatively, you can use Action object to set the tooltip:

```
exitAction.putValue(Action.SHORT_DESCRIPTION, "Exit");
```


GridBaglayout



GridBagLayout



GridBagLayout



GridBagLayout

- Create an object of type GridBagLayout.
- You don't need to tell it how many rows and columns the underlying grid has.
- Instead, the layout manager will try to guess it from the information you give it later.



GridBagLayout

- Create an object of type GridBagLayout.
- You don't need to tell it how many rows and columns the underlying grid has.
- Instead, the layout manager will try to guess it from the information you give it later.
- Set this GridBagLayout object to be the layout manager for the component.



GridBagLayout

- Create an object of type GridBagLayout.
- You don't need to tell it how many rows and columns the underlying grid has.
- Instead, the layout manager will try to guess it from the information you give it later.
- Set this GridBagLayout object to be the layout manager for the component.
- For each component, create an object of type GridBagConstraints.
- Set field values of the GridBagConstraints object to specify how the components are laid out within the grid bag.



GridBagLayout

- Create an object of type GridBagLayout.
- You don't need to tell it how many rows and columns the underlying grid has.
- Instead, the layout manager will try to guess it from the information you give it later.
- Set this GridBagLayout object to be the layout manager for the component.
- For each component, create an object of type GridBagConstraints.
- Set field values of the GridBagConstraints object to specify how the components are laid out within the grid bag.
- Finally, add each component with its constraints by using the call:

`add(component, constraints);`



GridBagLayout Example

```
GridBagLayout layout = new GridBagLayout();  
panel.setLayout(layout);  
GridBagConstraints constraints = new GridBagConstraints();  
  
constraints.weightx = 100;  
constraints.weighty = 100;  
constraints.gridx = 0;  
constraints.gridy = 2;  
constraints.gridwidth = 2;  
constraints.gridheight = 1;  
panel.add(component, constraints);
```


Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor